Computational Gastronomy — Assignment 2

Question 1: Frequent Item Set Mining (FISM)

Monsoon 2026

Problem Statement

Using the data of 10,000 recipes and their ingredients obtained for Assignment 1, implement any of the 'frequent item set mining (FISM) algorithms':

- **(a)** Provide a complete list of item sets of size one, two, and three. `[5]`
- **(b)** List the top 20 item sets of size one, two, and three. `[3]`
- **(c)** Define what are the 'support' & 'confidence' in the context of item set mining. `[2]`

In [1]:
```
import sys
import os
import time
from collections import Counter
from itertools import combinations
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Display options
pd.set_option('display.max_columns', 20)
pd.set_option('display.width', 160)
pd.set_option('display.max_colwidth', 60)

DATA_PATH = Path('../Group_23_Assignment1/Q1b_ingredients_ner.csv')
print(f"Loading ingredient data from: {DATA_PATH}")
```

```
Loading ingredient data from: ../Group_23_Assignment1/Q1b_ingredients_ner.csv
```

In [2]:
```
# Load the extracted NER ingredients from Assignment 1
ner_df = pd.read_csv(DATA_PATH, keep_default_na=False)

# Filter out empty ingredient names
valid_ner = ner_df[ner_df['Ingredient Name'].str.strip() != '']

# Group by Recipe ID to extract the basket / itemset of unique ingredients per
recipe_baskets = valid_ner.groupby('Recipe ID')['Ingredient Name'].apply(lambda

N = len(recipe_baskets)
print(f"Total recipes (transactions N): {N:,}")
print(f"Total valid ingredient mentions: {len(valid_ner):,}")
unique_vocab = sorted(list(set(valid_ner['Ingredient Name'])))
print(f"Vocabulary of unique ingredients (|I|): {len(unique_vocab):,}")

# Display sample recipe basket
sample_id = list(recipe_baskets.keys())[0]
```

Loading [MathJax]/extensions/Safe.js

```
print(f"Sample Recipe (ID {sample_id}) ingredients ({len(recipe_baskets[sample_
print(recipe_baskets[sample_id])
```

```
Total recipes (transactions N): 10,000
Total valid ingredient mentions: 100,504
Vocabulary of unique ingredients (|I|): 2,425
Sample Recipe (ID 1) ingredients (13 items):
['baby corn', 'coriander', 'cumin seed', 'garlic clove', 'kiwi', 'olive oil',
'red cabbage', 'red onion', 'red pepper', 'smoked paprika', 'tomato', 'vegan b
ouillon', 'wholemeal flour']
```

Algorithm Implementation: Apriori Frequent Itemset Mining

We implement the classic **Apriori Algorithm** (Agrawal & Srikant, 1994) from first principles. The algorithm exploits the **Apriori Principle** (anti-monotonicity of support):

> *All non-empty subsets of a frequent itemset must also be frequent.*

This allows efficient candidate pruning at each level k:

1. **Level 1 ($k=1$):** Count frequencies of all individual ingredients across the $N=10,000$ recipes. Retain those with $\text{count} \ge \text{min_count}$ (support $\ge \text{min_sup}$).
2. **Level 2 ($k=2$):** Generate candidate pairs from frequent 1-itemsets and count their co-occurrences in recipe baskets. Retain pairs with $\text{count} \ge \text{min_count}$.
3. **Level 3 ($k=3$):** Generate candidate triplets whose 2-item subsets are all verified to be frequent (Apriori candidate pruning). Count their occurrences and filter by minimum support.

We set a standard threshold of $\text{min_sup} = 0.01$ (1%, i.e., at least 100 recipes).

In [3]:
```
MIN_SUP = 0.01
MIN_COUNT = int(MIN_SUP * N)
print(f"Minimum Support Threshold: {MIN_SUP:.2%} ({MIN_COUNT} recipes)")

# ----------------- Level 1: 1-itemsets -----------------
t0 = time.time()
c1 = Counter()
for basket in recipe_baskets.values():
    for item in basket:
        c1[item] += 1

# Filter by min_count
freq_1 = {item: count for item, count in c1.items() if count >= MIN_COUNT}
f1_set = set(freq_1.keys())
t1 = time.time()
print(f"Found {len(freq_1):,} frequent 1-itemsets in {t1 - t0:.3f}s")

# ----------------- Level 2: 2-itemsets -----------------
```

Loading [MathJax]/extensions/Safe.js

```
for basket in recipe_baskets.values():
    # Filter basket items to only those that are frequent 1-items
    valid_items = [item for item in basket if item in f1_set]
    if len(valid_items) >= 2:
        for pair in combinations(valid_items, 2):
            c2[pair] += 1

freq_2 = {pair: count for pair, count in c2.items() if count >= MIN_COUNT}
f2_set = set(freq_2.keys())
t2 = time.time()
print(f"Found {len(freq_2):,} frequent 2-itemsets in {t2 - t1:.3f}s")

# ---------------- Level 3: 3-itemsets ----------------
c3 = Counter()
for basket in recipe_baskets.values():
    valid_items = [item for item in basket if item in f1_set]
    if len(valid_items) >= 3:
        for triplet in combinations(valid_items, 3):
            # Apriori pruning: all 3 size-2 subsets must be frequent
            if ((triplet[0], triplet[1]) in f2_set and
                (triplet[0], triplet[2]) in f2_set and
                (triplet[1], triplet[2]) in f2_set):
                c3[triplet] += 1

freq_3 = {triplet: count for triplet, count in c3.items() if count >= MIN_COUN
t3 = time.time()
print(f"Found {len(freq_3):,} frequent 3-itemsets in {t3 - t2:.3f}s")
print(f"Total mining time: {t3 - t0:.3f}s")
```

```
Minimum Support Threshold: 1.00% (100 recipes)
Found 210 frequent 1-itemsets in 0.009s
Found 412 frequent 2-itemsets in 0.046s
Found 151 frequent 3-itemsets in 0.086s
Total mining time: 0.141s
```

Q1(a) — Complete List of Item Sets of Size One, Two, and Three

We convert each level of mined frequent itemsets into structured DataFrames and save the complete lists to CSV files.

In [4]:

```
# 1-itemsets DataFrame
df_itemsets_1 = pd.DataFrame([
    {'Itemset': item, 'Support_Count': count, 'Support': count / N}
    for item, count in sorted(freq_1.items(), key=lambda x: x[1], reverse=True
])
df_itemsets_1.to_csv('Q1a_itemsets_size_1_complete.csv', index=False)

# 2-itemsets DataFrame
df_itemsets_2 = pd.DataFrame([
    {'Itemset': f"{pair[0]}, {pair[1]}", 'Support_Count': count, 'Support': cou
    for pair, count in sorted(freq_2.items(), key=lambda x: x[1], reverse=True
])
df_itemsets_2.to_csv('Q1a_itemsets_size_2_complete.csv', index=False)

# 3-itemsets DataFrame
pd.DataFrame([
    {'Itemset': f"{triplet[0]}, {triplet[1]}, {triplet[2]}", 'Support_Count': (
```

Loading [MathJax]/extensions/Safe.js

```
    for triplet, count in sorted(freq_3.items(), key=lambda x: x[1], reverse=T
])
df_itemsets_3.to_csv('Q1a_itemsets_size_3_complete.csv', index=False)

print(f"Summary of Complete Frequent Itemsets (min_sup = {MIN_SUP:.1%}):")
print(f"  • Size 1: {len(df_itemsets_1):,} itemsets -> saved to Q1a_itemsets_s
print(f"  • Size 2: {len(df_itemsets_2):,} itemsets -> saved to Q1a_itemsets_s
print(f"  • Size 3: {len(df_itemsets_3):,} itemsets -> saved to Q1a_itemsets_s
```

```
Summary of Complete Frequent Itemsets (min_sup = 1.0%):
  • Size 1: 210 itemsets -> saved to Q1a_itemsets_size_1_complete.csv
  • Size 2: 412 itemsets -> saved to Q1a_itemsets_size_2_complete.csv
  • Size 3: 151 itemsets -> saved to Q1a_itemsets_size_3_complete.csv
```

Q1(b) — Top 20 Item Sets of Size One, Two, and Three

Below are the top 20 most frequent itemsets for each size, ranked by support count and support percentage.

In [5]:

```
top20_1 = df_itemsets_1.head(20).copy()
top20_1.index = range(1, 21)
top20_1['Support (%)'] = (top20_1['Support'] * 100).map('{:.2f}%'.format)

top20_2 = df_itemsets_2.head(20).copy()
top20_2.index = range(1, 21)
top20_2['Support (%)'] = (top20_2['Support'] * 100).map('{:.2f}%'.format)

top20_3 = df_itemsets_3.head(20).copy()
top20_3.index = range(1, 21)
top20_3['Support (%)'] = (top20_3['Support'] * 100).map('{:.2f}%'.format)

# Save top 20 tables
top20_1.to_csv('Q1b_top20_itemsets_size_1.csv', index_label='Rank')
top20_2.to_csv('Q1b_top20_itemsets_size_2.csv', index_label='Rank')
top20_3.to_csv('Q1b_top20_itemsets_size_3.csv', index_label='Rank')

print("="*65)
print("TOP 20 ITEMSETS OF SIZE 1 (Individual Ingredients)")
print("="*65)
print(top20_1[['Itemset', 'Support_Count', 'Support (%)']].to_string())

print("\n" + "="*65)
print("TOP 20 ITEMSETS OF SIZE 2 (Ingredient Pairs)")
print("="*65)
print(top20_2[['Itemset', 'Support_Count', 'Support (%)']].to_string())

print("\n" + "="*65)
print("TOP 20 ITEMSETS OF SIZE 3 (Ingredient Triplets)")
print("="*65)
print(top20_3[['Itemset', 'Support_Count', 'Support (%)']].to_string())
```

Loading [MathJax]/extensions/Safe.js

```
================================================================
TOP 20 ITEMSETS OF SIZE 1 (Individual Ingredients)
================================================================
                Itemset  Support_Count Support (%)
1         garlic clove           3047      30.47%
2            olive oil           2294      22.94%
3               butter           2011      20.11%
4                onion           1729      17.29%
5           plain flour           1677      16.77%
6          caster sugar           1020      10.20%
7             coriander           1014      10.14%
8   golden caster sugar            967       9.67%
9         vegetable oil            965       9.65%
10              tomato            937       9.37%
11         double cream            904       9.04%
12            red onion            874       8.74%
13                 eggs            853       8.53%
14                 milk            849       8.49%
15               ginger            843       8.43%
16                lemon            831       8.31%
17            large egg            829       8.29%
18          icing sugar            825       8.25%
19               carrot            813       8.13%
20              parsley            809       8.09%

================================================================
TOP 20 ITEMSETS OF SIZE 2 (Ingredient Pairs)
================================================================
                        Itemset  Support_Count Support (%)
1       garlic clove, olive oil           1257      12.57%
2           garlic clove, onion           1088      10.88%
3           butter, plain flour            704       7.04%
4              olive oil, onion            648       6.48%
5          garlic clove, tomato            642       6.42%
6       coriander, garlic clove            626       6.26%
7          garlic clove, ginger            556       5.56%
8    garlic clove, rapeseed oil            478       4.78%
9         garlic clove, parsley            474       4.74%
10      garlic clove, red onion            451       4.51%
11                onion, tomato            438       4.38%
12           olive oil, parsley            424       4.24%
13    garlic clove, red chilli             421       4.21%
14            olive oil, tomato            415       4.15%
15                 butter, milk            413       4.13%
16         butter, garlic clove            397       3.97%
17         carrot, garlic clove            388       3.88%
18                carrot, onion            380       3.80%
19  garlic clove, vegetable oil            376       3.76%
20          garlic clove, thyme            368       3.68%

================================================================
TOP 20 ITEMSETS OF SIZE 3 (Ingredient Triplets)
================================================================
                                   Itemset  Support_Count Support (%)
1          garlic clove, olive oil, onion            460       4.60%
2             garlic clove, onion, tomato            358       3.58%
3         garlic clove, olive oil, tomato            307       3.07%
4          coriander, garlic clove, onion            275       2.75%
5          clove, olive oil, parsley                 273       2.73%
6        garlic clove, olive oil, parmesan           243       2.43%
```

Loading [MathJax]/extensions/Safe.js

7	coriander, garlic clove, ginger	242	2.42%
8	coriander, cumin, garlic clove	234	2.34%
9	garlic clove, onion, tomato puree	234	2.34%
10	carrot, garlic clove, onion	231	2.31%
11	garlic clove, ginger, onion	217	2.17%
12	olive oil, onion, tomato	197	1.97%
13	coriander, garlic clove, tomato	192	1.92%
14	garlic clove, olive oil, red onion	188	1.88%
15	garlic clove, onion, rapeseed oil	188	1.88%
16	garlic clove, onion, parsley	178	1.78%
17	carrot, celery, onion	175	1.75%
18	garlic clove, olive oil, thyme	174	1.74%
19	garlic clove, olive oil, tomato puree	171	1.71%
20	butter, milk, plain flour	169	1.69%

Q1(c) — Definitions of 'Support' & 'Confidence' in Item Set Mining

1. Support

- **Definition**: The **support** of an itemset $X \subseteq I$ is the proportion (fraction or percentage) of transactions in the dataset D that contain the entire itemset X. $$\text{Support}(X) = \frac{\sigma(X)}{|D|} = \frac{|\{T \in D \mid X \subseteq T\}|}{|D|}$$ where $\sigma(X)$ represents the support count (frequency of recipes containing X), and $|D| = N$ is the total number of transactions (recipes).
- **Significance**: Support measures the statistical significance, prevalence, or popularity of an itemset. Filtering by a minimum support threshold ($ ext{min_sup}$) ensures that rules are not derived from rare or coincidental co-occurrences.
- **Example from our data**: The pair $X = \{\text{garlic clove}, \text{olive oil}\}$ appears in 1,257 recipes out of 10,000, giving: $$\text{Support}(\{\text{garlic clove}, \text{olive oil}\}) = \frac{1257}{10000} = 0.1257 \quad (12.57\%)$$

---

2. Confidence

- **Definition**: The **confidence** of an association rule $X \rightarrow Y$ (where $X, Y \subseteq I$ and $X \cap Y = \emptyset$) is the conditional probability that a transaction contains itemset Y, given that it already contains itemset X. $$\text{Confidence}(X \rightarrow Y) = P(Y \mid X) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)} = \frac{\sigma(X \cup Y)}{\sigma(X)}$$
- **Significance**: Confidence measures the predictive power, reliability, or certainty of an inference. It quantifies how often the rule is found to be true among all instances where the antecedent X is present.
- **Example from our data**: Consider the rule: $$\{\text{onion}\} \rightarrow \{\text{garlic clove}\}$$ Support count of $\{\text{onion}\} = 1,729$, and co-occurrence count of $\{\text{onion}, \text{garlic clove}\} = 1,088$. $$\text{Confidence}(\{\text{onion}\} \rightarrow \{\text{garlic clove}\}) = \frac{1088}{1729} \approx 0.6293 \quad (62.93\%)$$ This means that in nearly 63% of recipes containing onion, garlic clove is also used.

Loading [MathJax]/extensions/Safe.js

In [6]:
```
# Example demonstration of association rules confidence from top pairs
sample_rules = []
for pair, count in sorted(freq_2.items(), key=lambda x: x[1], reverse=True)[:1(
    A, B = pair
    sup_AB = count / N
    sup_A = freq_1[A] / N
    sup_B = freq_1[B] / N

    # Rule A -> B
    conf_A_B = count / freq_1[A]
    # Rule B -> A
    conf_B_A = count / freq_1[B]

    sample_rules.append({'Rule': f"{A} -> {B}", 'Support': f"{sup_AB:.2%}", 'C(
    sample_rules.append({'Rule': f"{B} -> {A}", 'Support': f"{sup_AB:.2%}", 'C(

df_sample_rules = pd.DataFrame(sample_rules)
print("Sample Association Rules and Their Confidence:")
print(df_sample_rules.head(10).to_string(index=False))
```

```
Sample Association Rules and Their Confidence:
                    Rule Support Confidence
garlic clove -> olive oil  12.57%     41.25%
olive oil -> garlic clove  12.57%     54.80%
    garlic clove -> onion  10.88%     35.71%
    onion -> garlic clove  10.88%     62.93%
    butter -> plain flour   7.04%     35.01%
    plain flour -> butter   7.04%     41.98%
       olive oil -> onion   6.48%     28.25%
       onion -> olive oil   6.48%     37.48%
   garlic clove -> tomato   6.42%     21.07%
   tomato -> garlic clove   6.42%     68.52%
```

Question 1 Deliverables Summary

Deliverable File	Description	Question
`Q1a_itemsets_size_1_complete.csv`	Full list of mined frequent 1-itemsets (support $\ge 1\%$)	Q1(a)
`Q1a_itemsets_size_2_complete.csv`	Full list of mined frequent 2-itemsets (support $\ge 1\%$)	Q1(a)
`Q1a_itemsets_size_3_complete.csv`	Full list of mined frequent 3-itemsets (support $\ge 1\%$)	Q1(a)
`Q1b_top20_itemsets_size_1.csv`	Top 20 frequent 1-itemsets with support counts and percentages	Q1(b)
`Q1b_top20_itemsets_size_2.csv`	Top 20 frequent 2-itemsets with support counts and percentages	Q1(b)
`Q1b_top20_itemsets_size_3.csv`	Top 20 frequent 3-itemsets with support counts and percentages	Q1(b)

Loading [MathJax]/extensions/Safe.js